

Chapter 34. The Command Line, Linux & Networking Basics

Files, pipes, grep and awk, permissions, environment variables, shell scripts, SSH, and how a request finds a server.

Draft chapter

Contents

Chapter 34. The Command Line, Linux & Networking Basics

Why this matters

In plain English

34.1 Opening a terminal, and what you're looking at

34.2 Where am I, and what's here?

34.3 Pipes, redirection, and exit codes

34.4 Searching and summarizing text

34.5 Permissions: who can read, write, and run

34.6 Environment variables and PATH

34.7 Shell scripts

34.8 SSH: working on another machine

34.9 Networking in plain English

Common mistakes and how to spot them

In the real world: the disk that filled up at 2 a.m.

Tools

The project: a daily file that looks after itself

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 34. The Command Line, Linux & Networking Basics

Part III — Advanced Analytics & Analytics Engineering

Chapter at a glance

You will learn to: open a terminal on any operating system and find your way around · move, copy, and search files without a mouse · combine small tools with pipes to answer questions in seconds · search text with `grep`, and count and summarize it with `wc`, `sort`, `uniq`, `cut`, and `awk` · read and set file permissions · use environment variables and understand `PATH` · write a shell script that takes arguments, fails safely, and returns a meaningful exit code · connect to a server with SSH keys and copy files to it · explain IP addresses, DNS, ports, and HTTP in plain English, and use `curl` to test an API.

Before you start: Chapter 6 (you've opened a terminal and made a virtual environment) and Chapter 2 (files, formats, and what an API is). Chapter 28's database work is useful background but isn't required.

Time needed: 10–14 hours, spread over two weeks, most of it typing commands rather than reading.

Tools: a terminal running **bash** or **zsh**: Terminal on macOS, any terminal on Linux, and on Windows either **WSL** (the Windows Subsystem for Linux, recommended) or Git Bash. Windows differences are flagged throughout.

Practice data: the companion folder `ch34/`, which builds a `practice/` folder from Riverstone's 2025 sales: 61 daily export files, reference lists, and the report job's log. Every command in this chapter was run, and every output shown is the real output.

Why this matters

Every tool in the rest of this book lives in a terminal. `uv` and `pytest` (Chapter 29) are commands. `git` (Chapter 26) is a command. `dbt` (Chapter 32) is a command. Scheduled jobs (Chapter 20) are commands the machine runs at 3 a.m. with nobody watching. Cloud servers and containers (Chapter 49) have no desktop at all: a terminal is the only way in.

There's a second reason, and it shows up sooner. A surprising share of daily data work is faster in the terminal than anywhere else:

- "How many rows are in these 61 files, and which ones are empty?" One command.
- "The pipeline failed last night. What did it say?" `grep ERROR` on the log, in the time it takes a log viewer to load.

- "This 2 GB CSV crashes Excel and takes five minutes to load in pandas. I only need the rows for Pune." `grep Pune` finishes before pandas has started.
- "Which of these 400 files contains the customer id we're arguing about?" `grep -rL`.

You don't need to memorize the manual. You need about thirty commands, a clear picture of how they fit together, and the confidence to look up the rest.

In plain English

The terminal is a kitchen counter with a lot of small, sharp knives.

A graphical program is a food processor: one machine, a lid, a few buttons, and whatever it does is what you get. The terminal's tools each do one small thing, and you connect them. One tool finds lines; one counts them; one sorts them; one adds up a column. Alone they're almost too simple to be worth having. Connected with **pipes**, they answer questions that no single program on your machine can answer.

The counter has a few other fixtures:

- **A current folder.** Every command runs somewhere. Half of all early mistakes are "I was in the wrong place".
- **Three channels.** Every command takes **input**, produces **output**, and has a separate channel for **errors**, so you can keep the results and read the complaints separately.
- **A verdict.** Every command ends with a number: 0 means it worked. That number is how scheduled jobs know whether to raise an alarm.
- **Labels on the drawers.** **Environment variables** are named notes the shell hands to every program it starts: where to find things, which database to use, your API key.
- **A telephone line to other kitchens.** **SSH** gives you the same counter on a machine in another building, or another country.

34.1 Opening a terminal, and what you're looking at

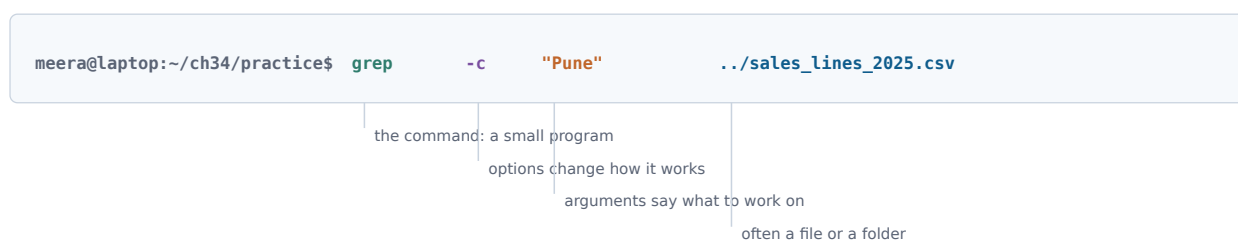
Your computer	What to open	Notes
macOS	Terminal (in Applications → Utilities), or iTerm2	The shell is zsh ; everything in this chapter works
Linux	Terminal, or <code>Ctrl+Alt+T</code> on most desktops	The shell is usually bash
Windows	WSL (Windows Subsystem for Linux): install from the Microsoft Store, or run <code>wsl --install</code> in PowerShell as administrator	Gives you a real Ubuntu with all these commands. The recommended route
Windows, alternative	Git Bash , installed with Git (Chapter 6)	Most commands here work; a few (<code>ss</code> , <code>apt</code>) don't

Your computer	What to open	Notes
Windows, native	PowerShell	Different commands (<code>Get-ChildItem</code> instead of <code>ls</code> , and so on). Fine for Windows administration, but the data world writes bash

The **shell** is the program that reads what you type, runs it, and shows the result. Bash and zsh are shells; the differences rarely matter for this chapter.

What you see when it opens is the **prompt**, something like `meera@laptop:~/ch34$` . It usually names the user, the machine, and the current folder. `~` is short for your **home folder**. Throughout this chapter, `$` marks a line you type; everything after it is the output.

Anatomy of a command



Output goes to the screen, errors go to a separate channel, and the command ends with an exit code:



Figure 34.1 — Every command is a program, some options, and some arguments, and it leaves behind an exit code.

The three keys that save the most time, before any command:

- **Tab** completes file and command names. Type `cd exp` and press Tab. Press it twice to list the possibilities.
- **Up arrow** brings back the previous command, so you can edit and rerun it. `history` lists them; `Ctrl+R` searches them.
- **Ctrl+C** stops a running command. (`Ctrl+D` says "no more input"; in an interactive shell that closes it.)

34.2 Where am I, and what's here?

Set up the practice folder first: copy `companion/ch34` somewhere convenient, then run its generator.

```
# terminal, in your copy of companion/ch34
$ python3 make_ch34_data.py
practice folder ready: 326 order lines, 23 customers, 61 daily export files
```

Now look around. Four commands do most of the navigating:

```
# terminal, in companion/ch34
$ cd practice

$ pwd
/home/meera/ch34/practice

$ ls
archive
exports
incoming
logs
reference

$ ls exports | head -3
orders_2025-11-01.csv
orders_2025-11-02.csv
orders_2025-11-03.csv

$ ls -l reference
total 8
-rw-r--r-- 1 meera meera 901 Jan  5 2026 customers.csv
-rw-r--r-- 1 meera meera 268 Jan  5 2026 products.csv
```

How it works:

- **cd** changes the current folder (*change directory*). `cd ..` goes up one level, `cd` on its own goes home, and `cd -` goes back where you were.
- **pwd** prints the current folder (*print working directory*). When a command surprises you, run `pwd` first.
- **ls** lists what's here. `ls -l` is the long form: permissions, owner, group, size in bytes, when it changed, and the name. `ls -lh` prints human-readable sizes (4.0K, 12M), and `ls -a` shows hidden files, whose names begin with a dot.
- Options combine: `ls -lha`. Most commands also accept long forms (`ls --all`), which are clearer in scripts.

Paths

A **path** says where something is. `/home/meera/ch34/practice/exports` is **absolute**: it starts at `/`, the root of the whole filesystem. `exports/orders_2025-12-16.csv` is **relative**: it starts from wherever you are now. Two shorthands appear everywhere: `.` is the current folder and `..` is the one above it.

Linux and macOS use `/` between folders; Windows uses `\`, but inside WSL and Git Bash you use `/` like everyone else. Names are **case-sensitive** on Linux: `Exports` and `exports` are different folders. That's one of the most common surprises when a script written on a Mac or Windows machine runs on a Linux server for the first time.

Looking inside files without opening an editor

```
# terminal, in companion/ch34/practice
$ head -2 exports/orders_2025-12-16.csv
order_id,order_date,customer_id,customer_name,city,segment,product_id,product_name,category,quantity,net_revenue,sales_rep
10168,2025-12-16,5,Metro Mart,Mumbai,Retail,101,Storage Box 10L,Storage,35,15050.00,Rahul Mehta

$ tail -2 exports/orders_2025-12-16.csv
10170,2025-12-16,21,Kitchen Kraft,Bengaluru,Retail,101,Storage Box 10L,Storage,35,15050.00,Rahul Mehta
10170,2025-12-16,21,Kitchen Kraft,Bengaluru,Retail,104,Food Container Set,Kitchen,35,21700.00,Rahul Mehta

$ wc -l exports/orders_2025-12-16.csv
8 exports/orders_2025-12-16.csv
```

- **head** shows the first lines (10 by default), **tail** the last. `tail -f somefile.log` follows a file as it grows, which is how you watch a job while it runs.
- **wc -l** counts lines (*word count*, `-l` for lines). Eight lines is seven order lines plus the header row.
- **cat** prints a whole file, and **less** pages through one: arrow keys and Page Down to move, `/pune` to search, `q` to quit. Use **less** for anything longer than a screen.

Watch out: **rm** does not use a recycle bin. `rm file.csv` deletes it, and `rm -r folder` deletes a folder and everything inside. There is no undo. Get in the habit of running `ls` with the same pattern first (`ls *.tmp`), looking at what it lists, and only then changing `ls` to `rm`. Never run `rm -rf` with a variable that might be empty: if `$DIR` is unset, `rm -rf $DIR/` is a command that has destroyed more than one production system.

Making, copying, moving, and removing

Command	What it does
<code>mkdir reports</code>	make a folder (<code>mkdir -p a/b/c</code> makes the whole chain)
<code>cp source.csv backup.csv</code>	copy a file (<code>cp -r folder1 folder2</code> copies a folder)
<code>mv old.csv new.csv</code>	move or rename: they're the same operation
<code>rm file.csv</code>	delete a file (<code>rm -r folder</code> deletes a folder)
<code>touch notes.txt</code>	create an empty file, or update its timestamp

Wildcards

The shell expands patterns *before* the command runs:

```
# terminal, in companion/ch34/practice
$ ls exports/orders_2025-12-0*.csv
exports/orders_2025-12-01.csv
exports/orders_2025-12-02.csv
exports/orders_2025-12-03.csv
exports/orders_2025-12-04.csv
exports/orders_2025-12-05.csv
exports/orders_2025-12-06.csv
exports/orders_2025-12-07.csv
exports/orders_2025-12-08.csv
exports/orders_2025-12-09.csv

$ ls exports/orders_2025-12-1?.csv | wc -l
10
```

* matches any number of characters, ? matches exactly one, and [12] matches either character listed. This is called **globbing**, and it's the shell's work, not the command's: `ls` only ever sees the list of names the shell found.

Finding files

`find` searches a folder tree by name, age, size, or type:

```
# terminal, in companion/ch34/practice
$ find . -name "orders_2025-12-2*.csv" | sort | head -3
./exports/orders_2025-12-20.csv
./exports/orders_2025-12-21.csv
./exports/orders_2025-12-22.csv

$ find . -type f -size +8k
./logs/report.log
```

`find . -type f -mtime -1` lists files changed in the last day, which is how you answer "did the export actually run last night?". `find . -name "*.tmp" -delete` cleans up, and is worth running once without `-delete` first.

34.3 Pipes, redirection, and exit codes

This is the idea that makes everything else worth learning.

Every command has three channels: **standard input** (where it reads from), **standard output** (where results go), and **standard error** (where complaints go). By default input comes from your keyboard, and both output channels go to the screen. You can point them somewhere else:

Symbol	Meaning
<code>command > file</code>	send output to a file, replacing it
<code>command >> file</code>	append output to a file
<code>command 2> errors.txt</code>	send only errors to a file
<code>command > out.txt 2>&1</code>	send output and errors to the same file

Symbol	Meaning
<code>command < file</code>	read input from a file
<code>command1 command2</code>	send command1's output straight into command2: a pipe

```
# terminal, in companion/ch34/practice
$ ls exports | wc -l
61

$ head -1 exports/orders_2025-12-16.csv > /tmp/header.csv

$ cat /tmp/header.csv
order_id,order_date,customer_id,customer_name,city,segment,product_id,product_name,category,quantity,net_revenue,sales_rep

$ ls missing_folder 2> /tmp/errors.txt

$ cat /tmp/errors.txt
ls: cannot access 'missing_folder': No such file or directory
```

Notice the third command printed nothing: the error went to the file instead of the screen. That's why a scheduled job writes `>> job.log 2>&1`: everything, results and complaints, ends up in one file with nothing lost.

tee does both at once: `./fetch_daily.sh 2025-12-16 | tee -a run.log` shows the output *and* appends it to a file.

Exit codes

Every command finishes with an **exit code**: 0 for success, anything else for failure. The shell keeps the last one in `$?`:

```
# terminal, in companion/ch34/practice
$ grep -q "Mumbai" reference/customers.csv; echo $?
0

$ grep -q "Kathmandu" reference/customers.csv; echo $?
1
```

`grep` returns 0 when it found something and 1 when it didn't, which is why `grep -q` (quiet) works so well inside `if` statements. Chapter 29's report command returns 1 when a month has no data; this is the same number, and it's what a scheduler checks.

Exit codes also chain commands:

- `command1 && command2` runs the second **only if** the first succeeded.
- `command1 || command2` runs the second **only if** the first failed.
- `command1 ; command2` runs both regardless.

```
# terminal, in companion/ch34/practice
$ test -f exports/orders_2025-12-16.csv && echo "file is there"
file is there

$ test -f exports/orders_2026-01-05.csv || echo "not published yet"
not published yet
```

34.4 Searching and summarizing text

Six small tools cover most data questions. The practice files hold one row per order line, with the same columns you know from the `sales_lines` view: order id, date, customer, city, segment, product, category, quantity, net revenue, and sales rep.

grep: find lines

```
# terminal, in companion/ch34/practice
$ grep -c "Pune" ../sales_lines_2025.csv
42

$ grep "Food Container Set" exports/orders_2025-12-16.csv
10169,2025-12-16,18,Evergreen Mart,Indore,Retail,104,Food Container Set,Kitchen,30,18600.00,Rahul Mehta
10170,2025-12-16,21,Kitchen Kraft,Bengaluru,Retail,104,Food Container Set,Kitchen,35,21700.00,Rahul Mehta

$ grep -i "sharma" ../sales_lines_2025.csv | head -2
10007,2025-01-24,1,Sharma Hardware,Mumbai,Retail,101,Storage Box 10L,Storage,45,19350.00,Neha Kulkarni
10007,2025-01-24,1,Sharma Hardware,Mumbai,Retail,102,Storage Box 25L,Storage,20,15000.00,Neha Kulkarni

$ grep -rl "Metro Mart" exports | sort | head -3
exports/orders_2025-11-26.csv
exports/orders_2025-12-16.csv
```

Option	Meaning
-c	count matching lines instead of printing them
-i	ignore case
-v	show lines that don't match
-r	search a whole folder tree
-l	print only the names of files that contain a match
-n	show line numbers
-w	match whole words only
-E	use extended regular expressions (Chapter 14's patterns)

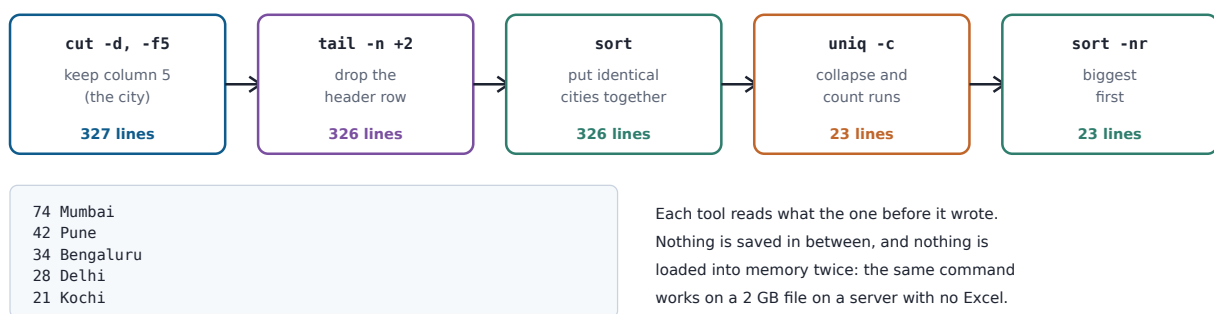
wc, sort, uniq, cut: count and group

```
# terminal, in companion/ch34/practice
$ cut -d, -f5 ../sales_lines_2025.csv | tail -n +2 | sort | uniq -c | sort -nr | head -5
 74 Mumbai
 42 Pune
 34 Bengaluru
 28 Delhi
 21 Kochi
```

How it works, one stage at a time:

1. **cut -d, -f5** takes field 5 of each line, using a comma as the delimiter: the city.
2. **tail -n +2** starts from line 2, dropping the header.
3. **sort** puts identical cities next to each other, because...
4. **uniq -c** collapses *adjacent* identical lines and counts them. Forgetting the **sort** before **uniq** is the classic mistake: it silently gives wrong counts.
5. **sort -nr** sorts numerically (**-n**), largest first (**-r**).
6. **head -5** keeps the top five.

One question, five small tools: order lines by city



Forgetting sort before uniq -c is the classic bug: uniq only collapses lines that are already next to each other.

Figure 34.2 — A pipeline is five small programs, each reading what the one before it wrote.

Mumbai leads with 74 order lines, and Pune's 42 matches the **grep -c "Pune"** count above. ✓ That is Chapter 13's "group by city and count" written with five tiny programs instead of one query, and it works on a file that no database has ever seen.

Watch out: **sort** sorts text unless you tell it otherwise. Without **-n**, **10** comes before **9**, because it compares character by character. And sort order depends on your language settings, which is why the verification for this chapter sets **LC_ALL=C**: it gives the plain byte order that everyone's machine agrees on.

awk: arithmetic on columns

awk reads a file line by line, splits it into fields, and runs your rules on each one. Two patterns cover nearly everything an analyst needs:

```
# terminal, in companion/ch34/practice
$ awk -F, 'NR > 1 { total += $11 } END { printf "%.2f\n", total }' exports/orders_2025-12-*.csv
439823.50

$ awk -F, 'NR > 1 { revenue[$9] += $11 } END { for (c in revenue) printf "%-12s %12.2f\n", c, revenue[c] }' exports/orders_2025-12-*.csv | sort -k2 -nr
Storage          220363.50
Kitchen          142180.00
Industrial        77280.00
category          0.00

$ awk -F, 'FNR > 1 { revenue[$9] += $11 } END { for (c in revenue) printf "%-12s %12.2f\n", c, revenue[c] }' exports/orders_2025-12-*.csv | sort -k2 -nr
Storage          220363.50
Kitchen          142180.00
Industrial        77280.00
```

The first version has a bug worth seeing: a line for `category`, awk's name for the header of every file after the first. `NR` counts lines across *all* the files, so `NR > 1` skips only the very first header. `FNR` restarts at each file, so `FNR > 1` skips all 31 of them. (The totals are unaffected, because awk treats the text `net_revenue` as 0 in arithmetic, which is exactly the kind of silent nonsense to watch for.)

- `-F,` sets the field separator to a comma. Fields are `$1`, `$2`, ..., and `$0` is the whole line.
- `NR > 1` is a condition: `NR` is the record (line) number, so this skips the first header.
- `{ total += $11 }` runs for every matching line; `END { ... }` runs once at the end.
- The second command uses an **associative array** (`revenue[$9]`), which is awk's version of a dictionary: sum by category in one pass.

Both numbers reconcile with the database. December's total of ₹439,823.50 is the figure Chapter 29's report produces, and the category split matches its Categories sheet: Storage ₹220,363.50, Kitchen ₹142,180.00, Industrial ₹77,280.00. ✓

Watch out: these tools don't understand CSV, only text. `cut -d,` and `awk -F,` split on every comma, including one inside `"Sharma Hardware, Mumbai"`. The practice files have no commas inside fields, so this works, and much real data is the same. When fields may contain commas, quotes, or line breaks, use a real CSV tool: `csvkit` and `xsv` / `qsv` are command-line tools that parse CSV properly, and `pandas` (Chapter 17) is right there. Use the shell for a quick look and a quick count; use a parser for anything you'll publish.

Reading logs

This is where `grep` earns its keep. The practice folder has the report job's log for November and December:

```
# terminal, in companion/ch34/practice
$ wc -l logs/report.log
187 logs/report.log

$ grep -c ERROR logs/report.log
3

$ grep ERROR logs/report.log
2025-11-09 07:00:04 ERROR riverstone_report: database connection refused
2025-12-07 07:00:04 ERROR riverstone_report: no sales lines for this day
2025-12-25 07:00:04 ERROR riverstone_report: no sales lines for this day

$ grep -A1 "connection refused" logs/report.log
2025-11-09 07:00:04 ERROR riverstone_report: database connection refused
2025-11-09 07:00:04 INFO riverstone_report: exit code 1

$ awk '{ print $3 }' logs/report.log | sort | uniq -c | sort -nr
180 INFO
  4 WARNING
   3 ERROR
```

Three failures in 61 days, two of them the same harmless cause (no orders on a Sunday and on Christmas Day), and one real problem on 9 November. `-A1` shows one line **after** each match (`-B` before, `-C` both), which is how you see what happened next. Finding that in a log viewer takes minutes; here it takes one line, and the same command works on a server over SSH where there is no log viewer at all.

34.5 Permissions: who can read, write, and run

Every file has an **owner**, a **group**, and nine permission bits. `ls -l` shows them:

```
# terminal, in companion/ch34/practice
$ ls -l reference/customers.csv
-rw-r--r-- 1 meera meera 901 Jan  5  2026 reference/customers.csv
```

Read the first column in four pieces: `-` (a regular file; `d` would be a folder, `l` a link), then three sets of three, for **user** (the owner), **group**, and **others** (everyone else). Each set is `r` read, `w` write, `x` execute, or `-` for "not allowed". So `rw-r--r--` means the owner can read and write, and everyone else can only read.

Reading ls -l

```
-rw-r--r-- 1 meera meera 901 Jan 5 2026 customers.csv
```

type: - file, d folder, l link
user: owner: read + write
group: read only
others: read only

Each set of three is read (r), write (w), execute (x).
On a folder, x means "can go into it".

644	rw- r-- r--	owner writes, everyone reads	data files, documents
755	rwX r-X r-X	owner writes, everyone reads and runs	scripts, folders
600	rw- --- ---	owner only	SSH keys, .env files with passwords
700	rwX --- ---	owner only, and can enter	~/ssh

chmod +x script.sh adds the execute bit; chmod 600 key removes everyone else's access.

Figure 34.3 – Nine bits, three audiences, and four numbers worth memorizing.

Each set is also a digit, adding read 4, write 2, and execute 1:

Number	Means	Common use
644	owner read and write; everyone else read	data files, documents
755	owner everything; everyone else read and run	scripts, folders
600	owner read and write; nobody else anything	SSH private keys, .env files with passwords
700	owner everything; nobody else anything	private folders such as ~/.ssh

On a folder, the bits mean something slightly different: **r** lists the names inside, **w** creates and deletes entries, and **x** lets you go into it or reach anything within it. A folder with no **x** is a locked cabinet: you can't get at the files even if their own permissions would allow it.

Watch a script go from unrunnable to runnable:

```
# terminal, in companion/ch34/practice
$ printf '#!/usr/bin/env bash\n echo "the export ran"\n' > run_export.sh

$ ls -l run_export.sh
-rw-rw-r-- 1 meera meera 42 Sep 17 18:48 run_export.sh

$ ./run_export.sh
bash: line 75: ./run_export.sh: Permission denied

$ chmod +x run_export.sh

$ ls -l run_export.sh
-rwxrwxr-x 1 meera meera 42 Sep 17 18:48 run_export.sh

$ ./run_export.sh
the export ran
```

chmod (change mode) sets permissions, either by adding and removing (chmod +x, chmod g-w, chmod o-rwx) or by number (chmod 755 run_export.sh). chown changes the owner, and usually needs administrator rights.

sudo, and when to be careful

`sudo` runs one command as the **superuser** (`root`), the account that can do anything: `sudo apt install rsync`. Use it for installing software and editing system files, never out of habit. If a command fails with *Permission denied*, the first question is "should I be allowed to do this?", not "how do I force it?". On a shared server, the answer is often that the file belongs to a service account and your script should be writing somewhere else.

Watch out: SSH refuses keys that other people can read. If your private key is `644`, `ssh` prints `WARNING: UNPROTECTED PRIVATE KEY FILE!` and refuses to use it. `chmod 600 ~/.ssh/id_ed25519` fixes it. The same logic applies to a `.env` file holding a database password (Chapter 29): `chmod 600`, and keep it out of Git.

34.6 Environment variables and PATH

An **environment variable** is a named value the shell hands to every program it starts. Chapter 29 used one to keep a database password out of the code; here's what's actually happening.

```
# terminal, in companion/ch34/practice
$ export RIVERSTONE_OUTPUT_DIR=/tmp/reports

$ echo "$RIVERSTONE_OUTPUT_DIR"
/tmp/reports

$ python3 -c 'import os; print(os.environ["RIVERSTONE_OUTPUT_DIR"])'
/tmp/reports

$ env | grep RIVERSTONE
RIVERSTONE_OUTPUT_DIR=/tmp/reports
```

- `export NAME=value` sets a variable and passes it to programs the shell starts. Without `export`, it stays in the shell itself.
- No spaces around `=`. `NAME = value` runs a command called `NAME`.
- `$NAME`, or better `"$NAME"` with quotes, reads it. The quotes matter: an unquoted value containing spaces is split into several arguments.
- `env` lists everything currently set; `unset NAME` removes one.
- Variables set this way last until you close the terminal. To make them permanent, put the `export` lines in `~/.bashrc` (bash) or `~/.zshrc` (zsh), which the shell reads when it starts. On a server, the scheduler or service manager supplies them instead.

PATH

`PATH` is the list of folders the shell searches, in order, when you type a command name:

```
# terminal, in companion/ch34/practice
$ echo "$PATH" | tr ':' '\n' | head -4
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin

$ which python3 grep
/usr/bin/python3
/usr/bin/grep

$ type cd
cd is a shell builtin
```

`which` shows which file would run. This is the answer to the two most common "but it works in my terminal" mysteries: a virtual environment puts its own folder at the front of `PATH` (that's all `activate` does), and a scheduled job runs with a much shorter `PATH` than your interactive shell, which is why `cron` jobs should call `/usr/bin/python3` or a full path to the environment rather than trusting `python3` to be found.

`type` explains what a name is: a file, a **built-in** command of the shell itself (like `cd`), an **alias**, or a function.

Watch out: never put secrets in your shell history. Typing `export DB_PASSWORD=hunter2` writes the password into `~/.bash_history`, which is readable by anyone with your account and is often backed up. Keep secrets in a `.env` file with `600` permissions, or in a secrets manager, and load them with `source .env` or a tool.

34.7 Shell scripts

A **shell script** is a file of commands. Anything you can type, you can save and run again, which is how a sequence of clever one-liners becomes something a colleague or a scheduler can use.

The companion folder has `daily_summary.sh`. Copy it into `practice/` and read it before running it:

```
# daily_summary.sh
#!/usr/bin/env bash
# Summarize one day's export: usage ./daily_summary.sh 2025-12-16
set -euo pipefail

DAY="${1:?usage: $0 YYYY-MM-DD}"
FILE="exports/orders_${DAY}.csv"

if [[ ! -f "$FILE" ]]; then
    echo "no export for $DAY" >&2
    exit 2
fi

LINES=$(( $(wc -l < "$FILE") - 1 ))
REVENUE=$(awk -F, 'FNR > 1 { total += $11 } END { printf "%.2f", total + 0 }' "$FILE")
CUSTOMERS=$(awk -F, 'FNR > 1 { seen[$4] = 1 } END { print length(seen) }' "$FILE")

echo "$DAY: $LINES order lines, $CUSTOMERS customers, revenue $REVENUE"
```

```
# terminal, in companion/ch34/practice
$ cp ../daily_summary.sh .

$ chmod +x daily_summary.sh

$ ./daily_summary.sh 2025-12-16
2025-12-16: 7 order lines, 3 customers, revenue 89800.00

$ ./daily_summary.sh 2025-12-25
2025-12-25: 0 order lines, 0 customers, revenue 0.00

$ ./daily_summary.sh 2026-01-05; echo "exit code $?"
no export for 2026-01-05
exit code 2

$ ./daily_summary.sh; echo "exit code $?"
./daily_summary.sh: line 6: 1: usage: ./daily_summary.sh YYYY-MM-DD
exit code 1
```

How it works:

- `#!/usr/bin/env bash`, the **shebang**, is the first line of every script: it tells the system which program runs the file. `env bash` finds bash wherever it's installed.
- `set -euo pipefail` belongs at the top of every script you write:
 - `-e` stop at the first command that fails, instead of carrying on with bad data,
 - `-u` treat an unset variable as an error, so a typo doesn't silently become an empty string (remember `rm -rf $DIR/`),
 - `-o pipefail` make a pipeline fail if **any** stage fails, not just the last one.
- `"${1:?message}"` takes the first argument, or exits with that message if it wasn't given. `$1`, `$2`, ... are the arguments, `$0` is the script's name, and `"$@"` is all of them.
- `$( ... )` is **command substitution**: run the command and use its output as text. `$(( ... ))` is arithmetic.
- `[[ ! -f "$FILE" ]]` is a test: *is this not a regular file?* Others: `-d` folder, `-s` non-empty file, `-z` empty string, `=` and `!=` for strings, `-eq` and `-lt` for numbers.

- **>&2** sends the message to standard error, so it doesn't pollute the script's real output when someone pipes it.
- **exit 2** returns a code that says *which* problem occurred: 0 success, 1 general failure, and your own numbers for the cases a caller might want to treat differently.
- **Always quote your variables:** `"$FILE"` , not `$FILE` . Without quotes, a path containing a space becomes two arguments, which is the single most common bug in shell scripts.

Loops

```
# terminal, in companion/ch34/practice
$ for day in 2025-12-15 2025-12-16 2025-12-17; do ./daily_summary.sh "$day"; done
2025-12-15: 0 order lines, 0 customers, revenue 0.00
2025-12-16: 7 order lines, 3 customers, revenue 89800.00
2025-12-17: 2 order lines, 1 customers, revenue 37628.00

$ for file in exports/orders_2025-12-2*.csv; do lines=$(( $(wc -l < "$file") - 1 )); [[ $lines -eq 0 ]] && echo "$file is empty"; done
exports/orders_2025-12-20.csv is empty
exports/orders_2025-12-22.csv is empty
exports/orders_2025-12-24.csv is empty
exports/orders_2025-12-25.csv is empty
exports/orders_2025-12-26.csv is empty
exports/orders_2025-12-27.csv is empty
exports/orders_2025-12-28.csv is empty
exports/orders_2025-12-29.csv is empty
```

A `for` loop over a wildcard is the workhorse of file processing: one command per file, in a predictable order. `while read line; do ... done < file.txt` reads a file line by line when you need that instead.

Watch out: shell scripts stop being the right tool sooner than you think. They're ideal for gluing commands together: fetch a file, check it, move it, call a program. Once you need data structures, CSV parsing, arithmetic with decimals, error handling with retries, or tests, write Python (Chapter 29). A 300-line bash script that nobody dares to change is a common and avoidable mess.

34.8 SSH: working on another machine

SSH (secure shell) gives you a terminal on a remote machine over an encrypted connection. It's how you reach the report server, a cloud database host, or a container.

Keys, not passwords

An **SSH key pair** is two matching files: a **private key** (`~/.ssh/id_ed25519`) that never leaves your machine, and a **public key** (`~/.ssh/id_ed25519.pub`) that you give to the server. The server uses the public key to check that you hold the private one. Nothing secret crosses the network, and there's no password to guess or reuse.

```
# terminal, on your own machine (run once)
$ ssh-keygen -t ed25519 -C "meera@laptop"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/meera/.ssh/id_ed25519):
Enter passphrase for "/home/meera/.ssh/id_ed25519" (empty for no passphrase):
Your identification has been saved in /home/meera/.ssh/id_ed25519
Your public key has been saved in /home/meera/.ssh/id_ed25519.pub

# copy the public key to the server (asks for your password this one time)
$ ssh-copy-id meera@reportserver.example.com
```

Choose a **passphrase**: it encrypts the private key, so a stolen laptop isn't a stolen server. `ssh-agent` remembers it for the session so you type it once a day.

Connecting and running commands

The examples below connect to a server named `reportserver`. In this chapter's test setup that name points at the same machine, which is a fine way to practice; on a real one it's a host somewhere else.

```
# terminal, in companion/ch34/practice
$ cat ~/.ssh/config
Host reportserver
    HostName 127.0.0.1
    User meera
    IdentityFile ~/.ssh/id_ed25519

$ ssh reportserver "hostname; ls /home/meera/ch34/practice/exports | wc -l"
vm
61
```

- `~/.ssh/config` gives a short name to a host, its address, the user, and which key to use, so `ssh reportserver` replaces `ssh -i ~/.ssh/id_ed25519 meera@10.4.2.17`.
- `ssh host "command"` runs one command and returns, which is exactly what a scheduled job or a deployment script needs. Without a command, you get an interactive shell; `exit` (or `Ctrl+D`) comes home.

Copying files

```
# terminal, in companion/ch34/practice
$ scp exports/orders_2025-12-16.csv reportserver:/tmp/

$ ssh reportserver "wc -l /tmp/orders_2025-12-16.csv"
8 /tmp/orders_2025-12-16.csv

$ rsync -a --delete exports/ reportserver:/tmp/exports_copy/

$ ssh reportserver "ls /tmp/exports_copy | wc -l"
61
```

- `scp` copies one file or folder, like `cp` with a machine name in front of the path.
- `rsync` copies only what differs, which matters when the folder is large or the connection is slow. `-a` keeps permissions and timestamps, `--delete` makes the destination match the source exactly, and `-n` shows what *would* happen without doing it. Run `rsync -an --delete ...` first, every time.

- The other direction works the same way: `scp reportserver:/var/log/report.log .` brings a file back.

Watch out: tunnels and ports. `ssh -L 5433:localhost:5432 reportserver` forwards your local port 5433 to the server's PostgreSQL. DBeaver on your laptop then connects to `localhost:5433` and reaches a database that isn't exposed to the internet at all. It's a common and legitimate pattern, and it's also something many companies audit, so use it with your platform team's blessing.

34.9 Networking in plain English

Four ideas cover most of what a data person needs.

IP addresses: which machine

An **IP address** identifies a machine on a network, like `142.250.183.4` (IPv4) or a longer IPv6 address. Some ranges are **private**, used inside company networks and never routed on the public internet: `10.x.x.x`, `172.16–31.x.x`, and `192.168.x.x`. And `127.0.0.1`, called **localhost**, always means *this machine*: the mock servers in Chapters 29 and 34 listen there, which is why nothing they do leaves your computer.

DNS: names into addresses

Typing an address is unworkable, so the **Domain Name System** translates names into addresses. Your machine asks a DNS server "*what is the address of `api.example.com` ?*" and caches the answer for a while.

```
# terminal, in companion/ch34/practice
$ getent hosts localhost
127.0.0.1      localhost
```

`dig api.example.com` or `nslookup api.example.com` does a full lookup and prints the result. DNS is the reason a database connection can fail with "*could not translate host name*": the name, not the database, is the problem. It's also how a company points `warehouse.internal` at a new server without changing a single connection string.

Ports: which program on that machine

One machine runs many services, so each listens on a numbered **port**. Some numbers are conventions: 22 SSH, 80 HTTP, 443 HTTPS, 5432 PostgreSQL, 3306 MySQL, 8080 and 8000 for test servers.

```
# terminal, in companion/ch34/practice
$ python3 ../daily_file_server.py > /tmp/server.log 2>&1 &

$ sleep 1; cat /tmp/server.log
daily file server on http://127.0.0.1:8034/exports/ (Ctrl+C to stop)

$ ss -ltn | grep 8034
LISTEN 0      5          127.0.0.1:8034      0.0.0.0:*
```

`ss -ltn` lists listening **t**CP sockets with **n**umeric addresses. (Older systems have `netstat -ltn`.) An address of `127.0.0.1:8034` means only this machine can connect; `0.0.0.0:8034` would mean anyone who can reach the machine. When something "can't connect", this command answers the first question: is anything listening at all?

Adding `&` to a command runs it in the **background** and gives you the prompt back. `jobs` lists background jobs, `fg` brings one forward, and `kill %1` stops it.

HTTP: asking for something

HTTP is the language of web requests: a **method** (GET to read, POST to send), a path, headers, and a body. The answer has a **status code** (Chapter 29's 200, 401, 429, 503) and usually a body. `curl` speaks it from the command line:

```
# terminal, in companion/ch34/practice
$ curl --silent http://127.0.0.1:8034/health
ok

$ curl --silent --output /dev/null --write-out "%{http_code}\n"
http://127.0.0.1:8034/exports/orders_2025-12-16.csv
200

$ curl --silent --output /dev/null --write-out "%{http_code}\n"
http://127.0.0.1:8034/exports/orders_2026-01-05.csv
404

$ curl --silent --head http://127.0.0.1:8034/exports/orders_2025-12-16.csv | grep -i "^content-"
Content-Type: text/csv
Content-Length: 834

$ curl --silent http://127.0.0.1:8034/exports/orders_2025-12-16.csv | wc -l
8

$ kill %1
```

What happens when curl asks for a file

```
curl --silent http://127.0.0.1:8034/exports/orders_2025-12-16.csv
```

`http`

the protocol: HTTP, or https for encrypted

`127.0.0.1`

which machine (localhost = this one)

`8034`

which program on it: the port

`/exports/orders_...`

what to ask for: the path

1. Name → address

A name like api.example.com is looked up in DNS. 127.0.0.1 is already an address, so there is nothing to look up.

2. Connect to the port

The machine's program listening on 8034 accepts the connection. ss -ltn shows who is listening.

3. Send the request

GET /exports/orders_2025-12-16.csv, plus headers such as an API key.

4. Read the answer

A status code (200 found, 404 missing, 429 slow down, 503 try later), headers, then the body.

Figure 34.4 – What a URL contains, and what happens after you press Enter.

- `--silent` hides the progress meter; `--output file` saves the body; `--write-out "%{http_code}"` prints just the status.
- `--head` asks for the headers only, which tells you the type and size before downloading 2 GB. (The other headers the server sends, including the date, are filtered out here with `grep` so the output is the same every time.)
- `--fail` makes curl return a non-zero exit code on a 4xx or 5xx status, which is what a script needs. `--max-time 30` sets a timeout. Both are in this chapter's project.
- For an API that needs a key: `curl -H "X-API-Key: demo-key" ...`, and `-d '{"json": "body"}' -H "Content-Type: application/json"` for a POST.

The 404 above is the same failure Chapter 29's client had to handle, seen from the other side: the file for 5 January 2026 hasn't been published, and no amount of retrying will change that today.

Try it. With the mock CRM API from Chapter 29 running, `curl --silent "http://127.0.0.1:8029/leads?page=1&page_size=3" -H "X-API-Key: demo-key"` shows the raw JSON that `CrMClient` parses, and dropping the header shows the 401.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Running a command in the wrong folder	<i>No such file or directory</i> for a file you can see in Finder or Explorer	<code>pwd</code> first; use Tab completion so wrong paths can't be typed
Forgetting that Linux is case-sensitive	A script that worked on your laptop fails on the server	Match the case exactly; keep file names lowercase

Mistake	Symptom	Fix
<code>rm</code> with a wildcard, or <code>rm -rf \$DIR/</code>	Files gone, with no recycle bin	Run <code>ls</code> with the same pattern first; <code>set -u</code> in scripts so an empty variable is an error
Unquoted variables in scripts	A path with a space becomes two arguments and half a file name disappears	Always <code>"\$VAR"</code>
<code>uniq</code> without <code>sort</code>	Counts too low, silently	<code>sort uniq -c</code>
<code>sort</code> on numbers without <code>-n</code>	9 comes after 10	<code>sort -n</code> , or <code>sort -k2 -nr</code> for a numeric column
<code>cut -d,</code> or <code>awk -F,</code> on real CSV	Wrong columns when a field contains a comma or a quote	Check the data; use <code>csvkit</code> , <code>qsv</code> , or <code>pandas</code> for anything published
<code>NR > 1</code> with several files in <code>awk</code>	Every file's header counted after the first	<code>FNR > 1</code>
Writing output over the input file (<code>sort f.csv > f.csv</code>)	An empty file	Write to a new name, then <code>mv</code>
Scheduled job "runs" but does nothing	Works by hand, not in cron	The job's <code>PATH</code> and environment are minimal: use full paths, set variables in the script
Missing shebang or execute bit	<i>Permission denied</i> , or the file opens as text	<code>#!/usr/bin/env bash</code> and <code>chmod +x</code>
No <code>set -euo pipefail</code>	The script carries on after a failure and writes a wrong file	Put it at the top of every script
Private key readable by others	WARNING: UNPROTECTED PRIVATE KEY FILE! and SSH refuses	<code>chmod 600 ~/.ssh/id_ed25519</code>
<code>curl</code> without <code>--fail</code> in a script	A 404 or 500 page saved as if it were data	<code>--fail --show-error --max-time N</code> , then check the exit code
Secrets typed on the command line	Passwords in <code>~/.bash_history</code> and in process lists	Use a <code>.env</code> file with <code>600</code> , or a secrets manager
Growing a shell script past a page or two	Nobody dares change it; no tests	Move to Python (Chapter 29) as soon as there's real logic
Assuming the shell parsed your pattern	<code>find . -name *.csv</code> fails oddly	Quote patterns for <code>find</code> : <code>find . -name "*.csv"</code>

In the real world: the disk that filled up at 2 a.m.

At 07:40 on a Tuesday, Meera's phone shows the alert Chapter 29's package now sends: the monthly report job exited with code 1. The log line is one she hasn't seen before: *ERROR riverstone_report: could not write reports/riverstone_monthly_2026-01.xlsx: No space left on device.*

The report server has no desktop. Everything below happens over SSH, and takes four minutes.

```
# terminal, on Meera's laptop
$ ssh reportserver
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1       40G   40G    0 100% /

$ du -h --max-depth=1 /var/riverstone | sort -h | tail -4
1.2G    /var/riverstone/exports
2.1G    /var/riverstone/reports
34G     /var/riverstone/incoming
38G     /var/riverstone

$ ls -lh /var/riverstone/incoming | tail -3
-rw-r--r-- 1 reportjob reportjob 412M Feb  1 02:00 orders_2026-01-30.csv
-rw-r--r-- 1 reportjob reportjob 412M Feb  2 02:00 orders_2026-01-31.csv
-rw-r--r-- 1 reportjob reportjob 412M Feb  3 02:00 orders_2026-02-01.csv

$ ls /var/riverstone/incoming | wc -l
84
```

Eighty-four daily files, 412 MB each, going back to November. The download step ran every night; nothing ever moved them out or deleted them. The archive folder the original script was supposed to use is empty, because that line was never written.

She clears just enough to let the report run, checking before deleting:

```
# terminal, on the report server
$ find /var/riverstone/incoming -name "orders_2025-*.csv" | wc -l
61
$ find /var/riverstone/incoming -name "orders_2025-*.csv" -print0 | xargs -0 du -ch | tail -1
25G    total
$ find /var/riverstone/incoming -name "orders_2025-*.csv" -delete
$ df -h / | tail -1
/dev/vda1      40G   15G   25G  38% /
```

The report runs by hand and goes out at 08:20. Then she fixes the cause, in the project below: a script that downloads the day's file, checks it, compresses it into the archive, and deletes the raw copy, with a monthly clean-up of archives older than a year. She adds one more line to the job's crontab: a weekly check that emails her when the disk is more than 80% full.

What she tells the data engineer: *"The job only ever added files. Downloads now get checked, compressed into the archive, and removed from `incoming`, which is 40 times smaller. There's a disk check on Mondays. And we should give `incoming` its own volume, so a runaway download can't take the whole server down with it."*

Three things made this fixable in minutes rather than a day: the failure was loud and had a reason (Chapter 29), the machine was reachable by SSH with a key, and `df`, `du`, `find`, and `ls` answer "what is filling the disk?" faster than any console.

Tools

Versions used for this chapter, checked in September 2026:

- **bash 5.2** on **Ubuntu 24.04**. macOS ships zsh, which behaves the same for everything here; the GNU tools on Linux have a few options that macOS's BSD versions don't (notably `ls --time-style` and `sed -i` syntax). Install the GNU versions on a Mac with `brew install coreutils gnu-sed` if you need them.
- **Windows: WSL 2** with Ubuntu, installed with `wsl --install` from an administrator PowerShell. Git Bash is the lighter alternative; PowerShell is a different language.
- **curl 8.5**, **OpenSSH 9.6**, **rsync 3.2**, **iproute2** (for `ss`), all standard on Ubuntu.
- **Python 3.12** for the practice data generator and the local file server.
- **Companion files** in `ch34/`: `make_ch34_data.py` (builds `practice/` from `sales_lines_2025.csv`), `daily_file_server.py` (the local file server), `daily_summary.sh` and `fetch_daily.sh` (the scripts), and `ch34_check.py` (checks the chapter's numbers).
- **Worth adding later:** `jq` for JSON on the command line, `ripgrep` (`rg`) as a faster `grep`, `fzf` for searching history, `tmux` for sessions that survive a dropped connection, and `qsv` or `csvkit` for proper CSV parsing.

The project: a daily file that looks after itself

Goal: a script that fetches one day's export, proves it arrived intact, archives it, logs what it did, and tells its caller what happened.

Option A: your own data. A file your team downloads, exports, or receives every day. Work in a folder you own, and keep credentials in environment variables, not in the script.

Option B: Riverstone. Use the companion `daily_file_server.py` as the partner's server, and write the script yourself before comparing with `fetch_daily.sh`.

What it must do:

1. Take the day as an argument, and reject anything that isn't `YYYY-MM-DD` with exit code 1.
2. Download `exports/orders_<day>.csv`, with a timeout, failing on a 404 rather than saving the error page. Exit 2 when the file isn't published yet.
3. Download the matching `.sha256` file and verify the download with `sha256sum --check`. Exit 3 on a mismatch, leaving the file for inspection.
4. Report what arrived: the number of order lines and the day's revenue (`wc -l` and `awk`).
5. Compress the file into `archive/` and remove the raw copy from `incoming/`.
6. Log every step, with a timestamp, to both the screen and `logs/fetch_daily.log`.
7. Exit 0 only when all of that worked.

Start the server in one terminal, then run the script in another:

```
# terminal, in companion/ch34/practice
$ cp ../fetch_daily.sh . && chmod +x fetch_daily.sh

$ python3 ../daily_file_server.py > /tmp/server.log 2>&1 &

$ sleep 1; ./fetch_daily.sh 2025-12-16 | cut -d' ' -f3-
INFO downloading orders_2025-12-16.csv from http://127.0.0.1:8034
INFO orders_2025-12-16.csv passed its checksum: 7 order lines, revenue 89800.00
INFO archived archive/orders_2025-12-16.csv.gz

$ ./fetch_daily.sh 2026-01-05 | cut -d' ' -f3-; echo "exit code ${PIPESTATUS[0]}"
curl: (22) The requested URL returned error: 404
INFO downloading orders_2026-01-05.csv from http://127.0.0.1:8034
ERROR could not download orders_2026-01-05.csv (not published yet, or the server is unreachable)
exit code 2

$ ls archive
orders_2025-12-16.csv.gz

$ ls incoming

$ zcat archive/orders_2025-12-16.csv.gz | awk -F, 'FNR > 1 { total += $11 } END { printf "%.2f\n",
total }'
89800.00

$ kill %1
```

(`cut -d' ' -f3-` drops the timestamp from each log line so the output is the same every time.
`${PIPESTATUS[0]}` is the exit code of the first command in the pipeline, not of `cut`.)

The compressed archive holds the same ₹89,800.00 the summary reported, so nothing was lost on the way in. ✓

Deliverables: the script, a run of each outcome (success, not published, bad checksum), and the log file.

Stretch goals:

- Add `--force` to re-fetch a day that's already archived, and refuse without it.
- Add a weekly clean-up that deletes archives older than 365 days (`find archive -name "*.gz" -mtime +365 -delete`), after running it once without `-delete` .
- Schedule it with `cron`: `crontab -e` , then `15 7 * * * cd /var/riverstone && ./fetch_daily.sh "$(date -d yesterday +%F)" >> logs/cron.log 2>&1` . Remember that `cron`'s `PATH` is minimal (section 34.6), and that `%` must be escaped in a crontab.
- Make the script fetch a range of days and keep going after a failure, reporting a summary at the end.
- Rewrite it in Python with `requests` (Chapter 29) and compare: which parts got simpler, which got longer?

You've got it when...

- ☐ I can find my way around a strange machine with `pwd` , `ls` , `cd` , and `find` , without a file manager.

- ☐ I can answer "how many rows, which files, which values" with a pipeline of small tools.
 - ☐ I know where output and errors go, and how to send them where I want.
 - ☐ I check exit codes, and my scripts return meaningful ones.
 - ☐ I can search a log for errors and see the lines around them.
 - ☐ I can read `ls -l` permissions and fix them with `chmod`.
 - ☐ I know what `PATH` is, and why a cron job can't find a command my terminal finds.
 - ☐ I can write a script that takes arguments, uses `set -euo pipefail`, quotes its variables, and fails safely.
 - ☐ I can connect to a server with an SSH key and copy files both ways.
 - ☐ I can explain IP, DNS, port, and HTTP status codes to a colleague, and test an API with `curl`.
-

Recap

- The **shell** runs commands in a **current folder**. `pwd`, `ls`, `cd`, `head`, `tail`, `less`, `cp`, `mv`, `rm`, `mkdir`, and `find` cover the file work; `Tab`, the up arrow, and `Ctrl+C` save the most time.
 - **Paths** are absolute (from `/`) or relative. Linux is case-sensitive. **Globbering** (`*`, `?`) is done by the shell before the command runs.
 - Every command has **standard input**, **standard output**, **standard error**, and an **exit code**. `>`, `>>`, `2>`, `<`, and `|` connect them; `&&` and `||` chain on success and failure.
 - **grep** finds lines; **wc**, **sort**, **uniq -c**, and **cut** count and group; **awk** does arithmetic on columns (`FNR > 1` skips each file's header). These tools see text, not CSV.
 - **Permissions** are user, group, and others, each with read, write, and execute: `644` data, `755` scripts, `600` secrets. `chmod` changes them.
 - **Environment variables** pass settings to programs; `PATH` decides which program a name runs; scheduled jobs get a minimal environment.
 - **Shell scripts** start with a **shebang** and `set -euo pipefail`, quote every variable, take arguments, and exit with a meaningful code. Move to Python when logic grows.
 - **SSH** uses a key pair; `~/.ssh/config` names hosts; `scp` and `rsync` copy files; keys must be `600`.
 - **Networking**: an **IP address** is a machine, **DNS** turns names into addresses, a **port** is a program on that machine, and **HTTP** is the request and its **status code**. `ss -ltn` shows what's listening; `curl` makes the request.
-

Practice exercises

Work in `companion/ch34/practice`. Predict each answer before you run it.

Warm-up

1. How many daily export files are there for November 2025, and how many lines do they hold in total (including headers)?
2. Which export file is the largest, and how many order lines does it hold?
3. Show the first two order lines of 26 November 2025 without opening an editor.
4. How many lines of `../sales_lines_2025.csv` mention Bengaluru, and how many mention `unassigned` ?

Core

5. List the ten customers with the most order lines in 2025, with their counts, using `cut` , `sort` , and `uniq` .
6. Total 2025 revenue from `../sales_lines_2025.csv` with `awk` , and check it against the ₹4,335,471 Chapter 13 reports.
7. Revenue by segment for December 2025, largest first.
8. Which days in December have no orders? Answer it twice: once with a `for` loop, once with `find` and `-size` .
9. Save the report job's `WARNING` lines to `/tmp/warnings.txt` and count them, without the count including the file name.
10. Write `weekly_summary.sh` , which takes a Monday's date and prints one line per day of that week using `daily_summary.sh` , then a total.

Stretch

11. Use `awk` to find the three order lines with the highest revenue in 2025, printing date, customer, product, and revenue.
12. Count the `curl` status codes you get for the first five days of January 2026 from the local file server, using a loop.
13. `scp` November's exports to `reportserver:/tmp/nov/` , then verify on the server that the line counts match, in a single `ssh` command.
14. A colleague's script runs `cd $DIR && rm -rf *` . Explain, precisely, what happens when `DIR` is unset, and rewrite the two lines safely.

Think about it (no commands needed)

15. Your script works in your terminal and fails in cron with `uv: command not found` . What's happening, and give two fixes.
 16. When is `grep` on a CSV the right tool, and when is it a trap? Give one example of each from this chapter's data.
-

Key terms

shell · bash · zsh · WSL · prompt · current working directory · path (absolute, relative) · home folder · globbing · standard input · standard output · standard error · redirection · pipe · exit code · `grep` · `wc` · `sort` · `uniq` · `cut` · `awk` · NR and FNR · field separator · `head` · `tail` · `less` · `find` · `chmod` · permission bits · owner · group · others · `sudo` · superuser · environment variable · `export` · `PATH` · shell built-in · shebang · `set -euo pipefail` · command substitution · quoting · `for` loop · argument (`$1`, `$@`) · background job · SSH · key pair · public key · private key · passphrase · `~/.ssh/config` · `scp` · `rsync` · port forwarding · IP address · localhost · private address range · DNS · port · HTTP · method · status code · `curl` · `ss` · `cron` · checksum · `sha256sum`

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 29, Python as Software, Not Scripts**, takes over when a shell script grows logic: it runs with the same exit codes, environment variables, and HTTP ideas you've just used.
 - **Chapter 26, The Professional Toolkit**, uses the terminal for Git.
 - **Chapter 32, Analytics Engineering with dbt**, is a command-line tool end to end.
 - **Chapter 20, Automating Reports & Delivering Insights**, schedules jobs with `cron` and its cloud equivalents.
 - **Chapter 46, Pipelines & Orchestration**, and **Chapter 49, Storage, Warehouses & Lakehouses**, run on machines you reach only this way.
 - **Chapter 50, Cloud Fundamentals**, builds on IP addresses, ports, and SSH keys for virtual machines and networks.
 - **Chapter 73, Data Engineering Question Bank**, asks about permissions, exit codes, and debugging a failed job.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1.

```
# terminal, in companion/ch34/practice
$ ls exports/orders_2025-11-*.csv | wc -l
30

$ cat exports/orders_2025-11-*.csv | wc -l
65
```

30 files, 30 headers plus 35 order lines. `wc -l exports/orders_2025-11-*.csv | tail -1` gives the same total with a `total` label, and also shows each file.

2.

```
# terminal, in companion/ch34/practice
$ ls -S exports | head -2
orders_2025-11-16.csv
orders_2025-12-16.csv

$ wc -l exports/orders_2025-11-16.csv
8 exports/orders_2025-11-16.csv
```

`ls -S` sorts by size, largest first. The file for 16 November is the biggest of the two months. (`du -h exports/* | sort -h | tail -2` is the more general form, since it works on folders too.)

3.

```
# terminal, in companion/ch34/practice
$ head -3 exports/orders_2025-11-26.csv | tail -2
10155,2025-11-26,3,Green Leaf Hotels,Pune,Hospitality,107,Lunch Box Set,Kitchen,10,3610.00,Farah Khan
10156,2025-11-26,5,Metro Mart,Mumbai,Retail,101,Storage Box 10L,Storage,15,6450.00,Rahul Mehta
```

`head -3` takes the header and the first two order lines; `tail -2` drops the header.

4.

```
# terminal, in companion/ch34/practice
$ grep -c Bengaluru ../sales_lines_2025.csv
34

$ grep -c unassigned ../sales_lines_2025.csv
16
```

34 lines mention Bengaluru, and 16 order lines have no sales rep recorded, which is Chapter 13's data-quality finding (11 orders, 10 of them not cancelled) seen at the line level.

5.

```
# terminal, in companion/ch34/practice
$ cut -d, -f4 ../sales_lines_2025.csv | tail -n +2 | sort | uniq -c | sort -nr | head -10
 32 Sharma Hardware
 32 Green Leaf Hotels
 29 Metro Mart
 25 Northgate Distributors
 25 Fresh Bowl Kitchens
 21 Harbour Traders
 21 Coastal Foods
 20 Deccan Packaging
 16 Spice Route Restaurants
 16 Evergreen Mart
```

Field 4 is the customer name. The same warning as before: this works because no customer name contains a comma.

6.

```
# terminal, in companion/ch34/practice
$ awk -F, 'FNR > 1 { total += $11 } END { printf "%.2f\n", total }' ../sales_lines_2025.csv
4335471.00
```

₹4,335,471.00, matching Chapter 13's total for 2025 to the rupee. ✓

7.

```
# terminal, in companion/ch34/practice
$ awk -F, 'FNR > 1 { revenue[$6] += $11 } END { for (s in revenue) printf "%-12s %12.2f\n", s, revenue[s] }' exports/orders_2025-12-*.csv | sort -k2 -nr
Wholesale      205238.50
Hospitality    131080.00
Retail         103505.00
```

The three segments add up to December's ₹439,823.50. ✓

8.

```
# terminal, in companion/ch34/practice
$ for f in exports/orders_2025-12-*.csv; do [[ $(wc -l < "$f") -eq 1 ]] && basename "$f"; done |
head -5
orders_2025-12-02.csv
orders_2025-12-03.csv
orders_2025-12-06.csv
orders_2025-12-07.csv
orders_2025-12-09.csv

$ find exports -name "orders_2025-12-*.csv" -size -125c | sort | wc -l
17
```

A file with only a header is 1 line and 124 bytes, so `-size -125c` (smaller than 125 bytes) finds them. The loop is clearer; the `find` version is shorter and depends on knowing the header's exact length, which is the kind of assumption that breaks when a column is added.

9.

```
# terminal, in companion/ch34/practice
$ grep WARNING logs/report.log > /tmp/warnings.txt

$ wc -l < /tmp/warnings.txt
4
```

`wc -l < file` reads from standard input, so `wc` never learns the file's name and prints only the number. `wc -l /tmp/warnings.txt` would print `4 /tmp/warnings.txt`, which is awkward inside `$( ... )`.

10.

```
#!/usr/bin/env bash
# weekly_summary.sh 2025-12-15
set -euo pipefail

MONDAY="${1:?usage: $0 YYYY-MM-DD (a Monday)}"
TOTAL=0

for offset in 0 1 2 3 4 5 6; do
    DAY=$(date -d "$MONDAY + $offset days" +%F)      # macOS: date -j -v+"$offset"d -f %Y-%m-%d
    "$MONDAY" +%F
    LINE=$(./daily_summary.sh "$DAY")
    echo "$LINE"
    REVENUE=${LINE##*revenue }
    TOTAL=$(awk -v a="$TOTAL" -v b="$REVENUE" 'BEGIN { printf "%.2f", a + b }')
done

echo "week of $MONDAY: revenue $TOTAL"
```

```
# terminal, in companion/ch34/practice
$ cp ../weekly_summary.sh . && chmod +x weekly_summary.sh

$ ./weekly_summary.sh 2025-12-15
2025-12-15: 0 order lines, 0 customers, revenue 0.00
2025-12-16: 7 order lines, 3 customers, revenue 89800.00
2025-12-17: 2 order lines, 1 customers, revenue 37628.00
2025-12-18: 2 order lines, 1 customers, revenue 23850.00
2025-12-19: 1 order lines, 1 customers, revenue 5750.00
2025-12-20: 0 order lines, 0 customers, revenue 0.00
2025-12-21: 1 order lines, 1 customers, revenue 2755.00
week of 2025-12-15: revenue 159783.00
```

`${LINE##*revenue }` strips everything up to the last "revenue ", one of the shell's parameter expansions. The total uses `awk` because bash's arithmetic is whole numbers only, which is a good sign that anything more complicated belongs in Python.

11.

```
# terminal, in companion/ch34/practice
$ awk -F, 'FNR > 1 { printf "%12.2f  %s  %-20s %s\n", $11, $2, $4, $8 }' ../sales_lines_2025.csv |
sort -nr | head -3
56700.00  2025-09-13  Deccan Packaging      Industrial Crate
51520.00  2025-12-08  Northgate Distributors Industrial Crate
51520.00  2025-10-13  Harbour Traders       Industrial Crate
```

Printing the sort key first and sorting numerically is the usual shell pattern for a top-N. The largest single order line is ₹56,700: 45 Industrial Crates to Deccan Packaging on 13 September. (Chapter 13's biggest *order*, ₹86,615 on 22 November, is several lines added together, which is the grain difference from Chapter 28 showing up in a shell pipeline.)

12.

```
# terminal, in companion/ch34/practice
$ python3 ../daily_file_server.py > /tmp/server.log 2>&1 &

$ sleep 1; for d in 01 02 03 04 05; do curl --silent --output /dev/null --write-out "%
{http_code}\n" "http://127.0.0.1:8034/exports/orders_2026-01-$d.csv"; done | sort | uniq -c
      5 404

$ kill %1
```

All five are 404: the server only has November and December 2025. A script that didn't check the status would have saved five files containing the words "not found".

13.

```
# terminal, in companion/ch34/practice
$ ssh reportserver "mkdir -p /tmp/nov"

$ scp exports/orders_2025-11-*.csv reportserver:/tmp/nov/

$ ssh reportserver "cat /tmp/nov/*.csv | wc -l"
65

$ cat exports/orders_2025-11-*.csv | wc -l
65
```

Both 65, so nothing was lost. For a large folder, `rsync -a --delete exports/ reportserver:/tmp/nov/` is the better tool, and `rsync -an --delete` shows what it would do first.

14. With `DIR` unset, `cd $DIR` expands to plain `cd`, which succeeds and moves to the home folder. The `&&` then runs `rm -rf *` there, deleting everything the user owns. Two changes make it safe:

```
set -euo pipefail                # -u makes an unset variable an error
cd "${DIR:?DIR is not set}" && rm -rf -- ./*
```

`"${DIR:?message}"` stops with a message if `DIR` is empty or unset, the quotes handle spaces, and `--` stops a file named like an option from being read as one. Better still, delete by pattern (`rm -f ./*.tmp`) rather than everything.

15. Cron runs with a minimal environment: a short `PATH` (often just `/usr/bin:/bin`), no `~/.bashrc`, and a different current folder. `uv` lives in your home folder, which isn't on that `PATH`. Two fixes: call it by full path (`/home/meera/.local/bin/uv run ...`), or set `PATH=` at the top of the crontab or the script. A third, often best: have cron run a small wrapper script that sets the environment explicitly, so the job doesn't depend on anyone's login setup.

16. `grep` is right for a quick look and a quick count on data you trust: *"how many of these 61 files mention Metro Mart, and which ones"* took one command and no parsing. It's a trap when the answer has to be correct in a published number and the data can contain commas, quotes, or newlines inside fields, or when the match could appear in the wrong column: `grep -c Storage ../sales_lines_2025.csv` counts lines where *any* field contains "Storage", including a product name and a category, which is not the same as "order lines in the Storage category". For that, use `awk -F, '$9 == "Storage"'`, or `pandas`.